```python
# ============================================================
# CELL 1: Mount Google Drive
# ============================================================
from google.colab import drive
drive.mount('/content/drive')

import os
SAVE_DIR = '/content/drive/MyDrive/AttentionChatbot'
os.makedirs(SAVE_DIR, exist_ok=True)
print(f"□ Drive mounted. Results will be saved to: {SAVE_DIR}")
```

```
Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
□ Drive mounted. Results will be saved to:
/content/drive/MyDrive/AttentionChatbot
```

```python
# ============================================================
# CELL 2: Install dependencies & imports
# ============================================================
!pip install torch torchvision matplotlib seaborn --quiet

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import numpy as np
import random
import re
import json
import time
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns
from collections import Counter
import warnings
warnings.filterwarnings('ignore')

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"□ Libraries loaded | Device: {device}")
```

```
□ Libraries loaded | Device: cuda
```

```python
# ============================================================
# CELL 3: Download Cornell Movie Dialogs Dataset
# ============================================================
!wget -q
http://www.cs.cornell.edu/~cristian/data/cornell_movie_dialogs_corpus.
zip
!unzip -q cornell_movie_dialogs_corpus.zip -d cornell_data
```

```
print("□ Dataset downloaded and extracted")
!ls cornell_data/cornell\ movie-dialogs\ corpus/
```

```
□ Dataset downloaded and extracted
chameleons.pdf               movie_lines.txt             README.txt
movie_characters_metadata.txt  movie_titles_metadata.txt
movie_conversations.txt        raw_script_urls.txt
```

```
# ================================================================
# CELL 4: Parse Cornell dataset → (input, output) pairs
# ================================================================

DATA_DIR = "cornell_data/cornell movie-dialogs\ corpus"

# Load lines
lines = {}
with open(f"cornell_data/cornell movie-dialogs
corpus/movie_lines.txt",
          encoding='iso-8859-1') as f:
    for line in f:
        parts = line.strip().split(' +++$+++ ')
        if len(parts) == 5:
            lines[parts[0]] = parts[4]

# Load conversations
conversations = []
with open(f"cornell_data/cornell movie-dialogs
corpus/movie_conversations.txt",
          encoding='iso-8859-1') as f:
    for line in f:
        parts = line.strip().split(' +++$+++ ')
        if len(parts) == 4:
            line_ids = eval(parts[3])
            conversations.append(line_ids)

# Build pairs
pairs = []
for conv in conversations:
    for i in range(len(conv) - 1):
        inp = lines.get(conv[i], "").strip()
        out = lines.get(conv[i+1], "").strip()
        if inp and out:
            pairs.append((inp, out))

print(f"□ Total conversation pairs: {len(pairs):,}")
print(f"\nSample pairs:")
for p in random.sample(pairs, 5):
    print(f"  IN : {p[0][:60]}")
    print(f"  OUT: {p[1][:60]}")
    print()
```

```
 Total conversation pairs: 221,282

Sample pairs:
  IN : Then you can learn to act normally. I had to!
  OUT: Aw, come on. You know you love me. You're a masochistic pain

  IN : She finds him.
  OUT: She sits down.

  IN : You clever bastard! I might have guessed. You only moved in
  OUT: Am I?

  IN : I wasn't. I wasn't. She was the pretty one. She died, why no
  OUT: So you left the Church because you had freckles?

  IN : I know.
  OUT: The police still have a lot of questions for Russell Nash.
```

```python
# ============================================================
# CELL 5: Clean text & build vocabulary
# ============================================================

MAX_LEN   = 15    # max tokens per sentence
MIN_COUNT = 3     # min word frequency to keep
MAX_PAIRS = 50000 # limit dataset size for training speed

PAD, SOS, EOS, UNK = 0, 1, 2, 3
PAD_TOKEN, SOS_TOKEN, EOS_TOKEN, UNK_TOKEN = '<PAD>', '<SOS>',
'<EOS>', '<UNK>'

def clean(text):
    text = text.lower().strip()
    text = re.sub(r"([.!?])", r" \1", text)
    text = re.sub(r"[^a-zA-Z.!?']+", r" ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

def tokenize(text):
    return clean(text).split()

# Filter by length
def filter_pair(p):
    return (1 <= len(tokenize(p[0])) <= MAX_LEN and
            1 <= len(tokenize(p[1])) <= MAX_LEN)

filtered = [p for p in pairs if filter_pair(p)]
random.shuffle(filtered)
filtered = filtered[:MAX_PAIRS]
print(f"  Filtered pairs: {len(filtered):,}")
```

```python
# Build vocabulary
counter = Counter()
for inp, out in filtered:
    counter.update(tokenize(inp))
    counter.update(tokenize(out))

vocab_words = [w for w, c in counter.items() if c >= MIN_COUNT]
word2idx = {PAD_TOKEN: PAD, SOS_TOKEN: SOS, EOS_TOKEN: EOS, UNK_TOKEN:
UNK}
for w in vocab_words:
    word2idx[w] = len(word2idx)
idx2word = {v: k for k, v in word2idx.items()}
VOCAB_SIZE = len(word2idx)
print(f"□ Vocabulary size: {VOCAB_SIZE:,}")

def encode(text, add_eos=True):
    tokens = tokenize(text)
    ids = [word2idx.get(t, UNK) for t in tokens]
    if add_eos:
        ids.append(EOS)
    return ids

# Save vocab
with open(f"{SAVE_DIR}/vocab.json", "w") as f:
    json.dump(word2idx, f)
print(f"□ Vocabulary saved to Drive")
```

```
□ Filtered pairs: 50,000
□ Vocabulary size: 8,696
□ Vocabulary saved to Drive
```

```python
# ============================================================
# CELL 6: PyTorch Dataset & DataLoader
# ============================================================

class DialogDataset(Dataset):
    def __init__(self, pairs):
        self.pairs = pairs

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):
        inp, out = self.pairs[idx]
        return encode(inp), encode(out)

def collate_fn(batch):
    batch.sort(key=lambda x: len(x[0]), reverse=True)
    src_seqs, tgt_seqs = zip(*batch)
```

```
    src_lens = [len(s) for s in src_seqs]
    tgt_lens = [len(t) for t in tgt_seqs]

    src_padded = torch.zeros(len(src_seqs), max(src_lens),
dtype=torch.long)
    tgt_padded = torch.zeros(len(tgt_seqs), max(tgt_lens),
dtype=torch.long)

    for i, (s, t) in enumerate(zip(src_seqs, tgt_seqs)):
        src_padded[i, :len(s)] = torch.tensor(s)
        tgt_padded[i, :len(t)] = torch.tensor(t)

    return src_padded, tgt_padded, src_lens, tgt_lens

split = int(0.9 * len(filtered))
train_ds = DialogDataset(filtered[:split])
val_ds   = DialogDataset(filtered[split:])

BATCH_SIZE = 64
train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE,
shuffle=True,
                          collate_fn=collate_fn)
val_loader   = DataLoader(val_ds,   batch_size=BATCH_SIZE,
shuffle=False,
                          collate_fn=collate_fn)
print(f"□ Train batches: {len(train_loader)} | Val batches:
{len(val_loader)}")
```

```
□ Train batches: 704 | Val batches: 79
```

```
# ============================================================
# CELL 7: Model Architecture
#   Embedding → LSTM Encoder → Attention → LSTM Decoder → Output
# ============================================================

class Encoder(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_layers,
dropout):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=PAD)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, n_layers,
                            batch_first=True, dropout=dropout if
n_layers > 1 else 0)
        self.dropout = nn.Dropout(dropout)

    def forward(self, src, src_lens):
        # src: (B, T)
        embedded = self.dropout(self.embedding(src))  # (B, T, E)
        packed = nn.utils.rnn.pack_padded_sequence(embedded, src_lens,
```

```
                                                        batch_first=True,
enforce_sorted=True)
        outputs, (hidden, cell) = self.lstm(packed)
        outputs, _ = nn.utils.rnn.pad_packed_sequence(outputs,
batch_first=True)
        # outputs: (B, T, H)  hidden/cell: (n_layers, B, H)
        return outputs, hidden, cell


class BahdanauAttention(nn.Module):
    """Additive (Bahdanau) attention"""
    def __init__(self, hidden_dim):
        super().__init__()
        self.W_enc = nn.Linear(hidden_dim, hidden_dim, bias=False)
        self.W_dec = nn.Linear(hidden_dim, hidden_dim, bias=False)
        self.v     = nn.Linear(hidden_dim, 1,          bias=False)

    def forward(self, decoder_hidden, encoder_outputs):
        # decoder_hidden : (B, H)
        # encoder_outputs: (B, T, H)
        energy = torch.tanh(
            self.W_enc(encoder_outputs) +             # (B, T, H)
            self.W_dec(decoder_hidden).unsqueeze(1)   # (B, 1, H)
        )                                             # (B, T, H)
        scores = self.v(energy).squeeze(-1)           # (B, T)
        weights = F.softmax(scores, dim=1)            # (B, T)
        context = torch.bmm(weights.unsqueeze(1),
                            encoder_outputs)          # (B, 1, H)
        return context.squeeze(1), weights            # (B, H), (B, T)


class Decoder(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_layers,
dropout):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=PAD)
        self.attention  = BahdanauAttention(hidden_dim)
        self.lstm = nn.LSTM(embed_dim + hidden_dim, hidden_dim,
n_layers,
                            batch_first=True, dropout=dropout if
n_layers > 1 else 0)
        self.fc_out = nn.Linear(hidden_dim * 2, vocab_size)
        self.dropout = nn.Dropout(dropout)

    def forward_step(self, token, hidden, cell, encoder_outputs):
        # token: (B,)
        embedded = self.dropout(self.embedding(token.unsqueeze(1)))  #
(B,1,E)
```

```
        attn_ctx, attn_wts = self.attention(hidden[-1],
encoder_outputs)
        lstm_in = torch.cat([embedded, attn_ctx.unsqueeze(1)], dim=2)
# (B,1,E+H)
        output, (hidden, cell) = self.lstm(lstm_in, (hidden, cell))
# (B,1,H)
        output = output.squeeze(1)
# (B,H)
        prediction = self.fc_out(torch.cat([output, attn_ctx], dim=1))
# (B,V)
        return prediction, hidden, cell, attn_wts


class Seq2SeqAttention(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_layers,
dropout):
        super().__init__()
        self.encoder = Encoder(vocab_size, embed_dim, hidden_dim,
n_layers, dropout)
        self.decoder = Decoder(vocab_size, embed_dim, hidden_dim,
n_layers, dropout)

    def forward(self, src, src_lens, tgt, teacher_forcing=0.5):
        B, T_tgt = tgt.shape
        enc_out, hidden, cell = self.encoder(src, src_lens)

        dec_input = torch.full((B,), SOS, dtype=torch.long,
device=device)
        outputs   = torch.zeros(B, T_tgt, VOCAB_SIZE, device=device)
        all_attn  = torch.zeros(B, T_tgt, src.size(1), device=device)

        for t in range(T_tgt):
            pred, hidden, cell, attn = self.decoder.forward_step(
                dec_input, hidden, cell, enc_out)
            outputs[:, t, :] = pred
            all_attn[:, t, :enc_out.size(1)] = attn
            # teacher forcing
            use_tf  = random.random() < teacher_forcing
            dec_input = tgt[:, t] if use_tf else pred.argmax(1)

        return outputs, all_attn


# ── Hyperparameters ──────────────────────────────────────────
EMBED_DIM  = 256
HIDDEN_DIM = 512
N_LAYERS   = 2
DROPOUT    = 0.3

model = Seq2SeqAttention(VOCAB_SIZE, EMBED_DIM, HIDDEN_DIM, N_LAYERS,
```

```
DROPOUT).to(device)
total_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
print(f"□ Model built | Trainable parameters: {total_params:,}")
print(model)
```

```
□ Model built | Trainable parameters: 22,295,544
Seq2SeqAttention(
  (encoder): Encoder(
    (embedding): Embedding(8696, 256, padding_idx=0)
    (lstm): LSTM(256, 512, num_layers=2, batch_first=True,
dropout=0.3)
    (dropout): Dropout(p=0.3, inplace=False)
  )
  (decoder): Decoder(
    (embedding): Embedding(8696, 256, padding_idx=0)
    (attention): BahdanauAttention(
      (W_enc): Linear(in_features=512, out_features=512, bias=False)
      (W_dec): Linear(in_features=512, out_features=512, bias=False)
      (v): Linear(in_features=512, out_features=1, bias=False)
    )
    (lstm): LSTM(768, 512, num_layers=2, batch_first=True,
dropout=0.3)
    (fc_out): Linear(in_features=1024, out_features=8696, bias=True)
    (dropout): Dropout(p=0.3, inplace=False)
  )
)
```

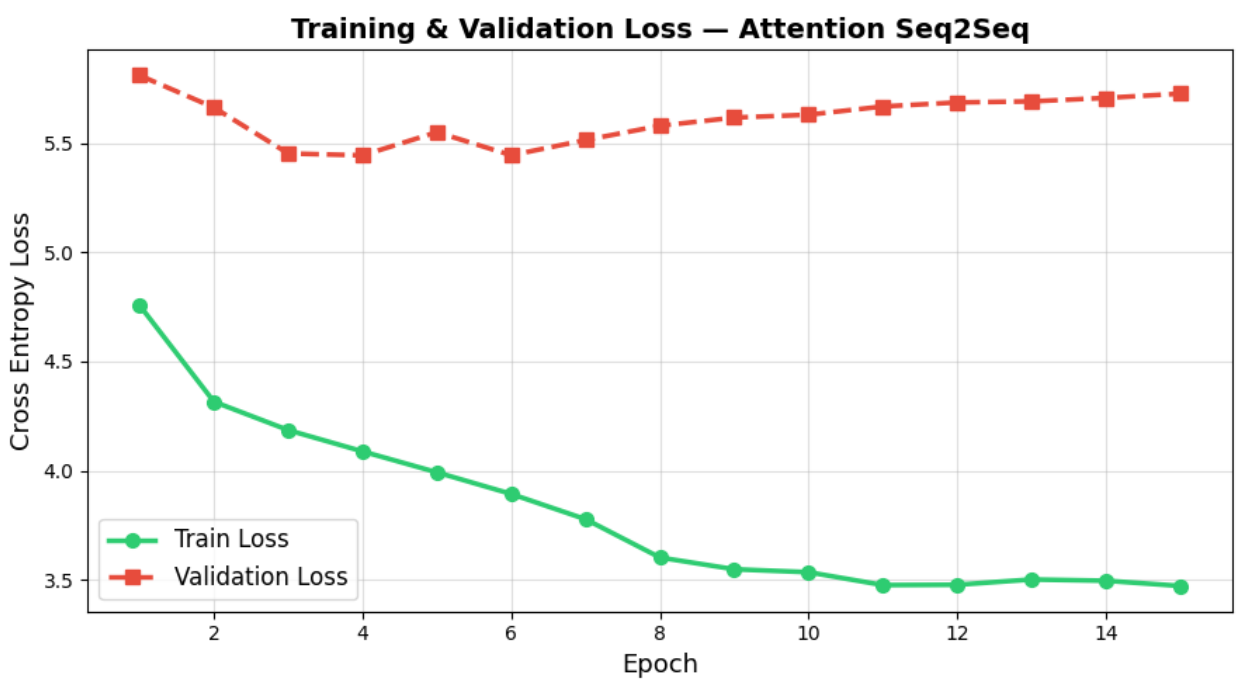
```
# ============================================================
# CELL 8: Training with CrossEntropyLoss + Adam
# ============================================================

EPOCHS      = 15
LR          = 0.001
CLIP        = 1.0

optimizer = torch.optim.Adam(model.parameters(), lr=LR)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer,
patience=2,
                                                       factor=0.5)
criterion = nn.CrossEntropyLoss(ignore_index=PAD)

def run_epoch(loader, train=True, tf_ratio=0.5):
    model.train() if train else model.eval()
    total_loss = 0
    ctx = torch.enable_grad() if train else torch.no_grad()
    with ctx:
        for src, tgt, src_lens, _ in loader:
            src, tgt = src.to(device), tgt.to(device)
            out, _ = model(src, src_lens, tgt,
```

```
teacher_forcing=tf_ratio if train else 0)
            # out: (B, T, V)  tgt: (B, T)
            loss = criterion(out.reshape(-1, VOCAB_SIZE),
tgt.reshape(-1))
            if train:
                optimizer.zero_grad()
                loss.backward()
                nn.utils.clip_grad_norm_(model.parameters(), CLIP)
                optimizer.step()
            total_loss += loss.item()
    return total_loss / len(loader)

train_losses, val_losses = [], []
best_val = float('inf')

print("=" * 55)
print(f"{'Epoch':>6} {'Train Loss':>12} {'Val Loss':>10} {'Time':>8}")
print("=" * 55)

for epoch in range(1, EPOCHS + 1):
    t0 = time.time()
    # anneal teacher forcing
    tf = max(0.3, 1.0 - epoch * 0.05)
    tr_loss = run_epoch(train_loader, train=True,  tf_ratio=tf)
    vl_loss = run_epoch(val_loader,   train=False)
    scheduler.step(vl_loss)
    train_losses.append(tr_loss)
    val_losses.append(vl_loss)
    elapsed = time.time() - t0
    flag = " ◀ best" if vl_loss < best_val else ""
    if vl_loss < best_val:
        best_val = vl_loss
        torch.save(model.state_dict(), f"{SAVE_DIR}/best_model.pt")
    print(f"{epoch:>6} {tr_loss:>12.4f} {vl_loss:>10.4f}
{elapsed:>7.1f}s{flag}")

print("=" * 55)
print(f"□ Best val loss: {best_val:.4f} | Model saved to Drive")
```

```
=======================================================
 Epoch   Train Loss   Val Loss     Time
=======================================================
     1       4.7569     5.8095    61.5s ◀ best
     2       4.3157     5.6623    61.2s ◀ best
     3       4.1855     5.4521    61.0s ◀ best
     4       4.0880     5.4435    61.5s ◀ best
     5       3.9923     5.5495    60.2s
     6       3.8932     5.4441    60.5s
     7       3.7775     5.5135    60.3s
     8       3.6023     5.5790    59.9s
```

```
     9        3.5485      5.6163     60.3s
    10        3.5350      5.6295     60.2s
    11        3.4761      5.6673     60.2s
    12        3.4775      5.6858     60.2s
    13        3.5018      5.6906     59.9s
    14        3.4960      5.7065     59.5s
    15        3.4722      5.7259     59.4s
============================================================
 Best val loss: 5.4435 | Model saved to Drive
```

```python
# ============================================================
# CELL 9: Plot & save training curves
# ============================================================

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(range(1, EPOCHS+1), train_losses, 'o-', color='#2ecc71',
        linewidth=2.5, markersize=7, label='Train Loss')
ax.plot(range(1, EPOCHS+1), val_losses,   's--', color='#e74c3c',
        linewidth=2.5, markersize=7, label='Validation Loss')
ax.set_xlabel('Epoch', fontsize=13)
ax.set_ylabel('Cross Entropy Loss', fontsize=13)
ax.set_title('Training & Validation Loss — Attention Seq2Seq',
fontsize=14, fontweight='bold')
ax.legend(fontsize=12)
ax.grid(True, alpha=0.4)
fig.tight_layout()
plt.savefig(f"{SAVE_DIR}/training_curves.png", dpi=150)
plt.show()
print(f" Training curves saved to Drive")
```



```
 Training curves saved to Drive

# ============================================================
# CELL 10: Inference — generate chatbot responses
# ============================================================

# Load best model
model.load_state_dict(torch.load(f"{SAVE_DIR}/best_model.pt",
map_location=device))
model.eval()

def respond(text, max_len=20):
    """Generate a reply and return (response_tokens,
attention_weights)."""
    tokens  = encode(text, add_eos=True)
    src_t   = torch.tensor([tokens], dtype=torch.long).to(device)
    src_len = [len(tokens)]

    with torch.no_grad():
        enc_out, hidden, cell = model.encoder(src_t, src_len)

    dec_input = torch.tensor([SOS], device=device)
    response  = []
    attn_hist = []

    with torch.no_grad():
        for _ in range(max_len):
            pred, hidden, cell, attn = model.decoder.forward_step(
                dec_input, hidden, cell, enc_out)
            top1 = pred.argmax(1).item()
            if top1 == EOS:
                break
            response.append(idx2word.get(top1, UNK_TOKEN))
            attn_hist.append(attn.squeeze(0).cpu().numpy())
            dec_input = torch.tensor([top1], device=device)

    attn_matrix = np.array(attn_hist) if attn_hist else
np.zeros((1,1))
    return " ".join(response), attn_matrix, tokenize(text)

# Quick test
test_inputs = [
    "how are you",
    "what is your name",
    "where are you going",
    "i love you",
    "what do you want",
    "are you okay",
    "tell me a story",
    "goodbye",
```

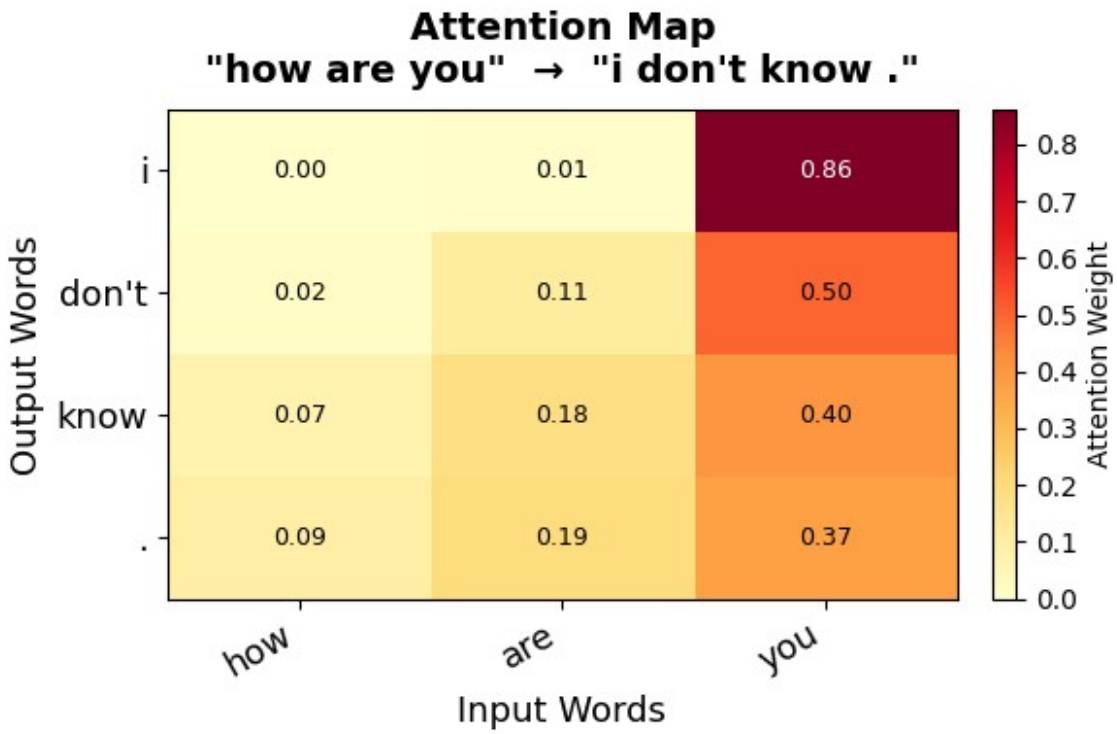
```
]

print("=" * 58)
print(f"{'INPUT':<28} {'RESPONSE'}")
print("=" * 58)
for q in test_inputs:
    reply, _, _ = respond(q)
    print(f"  {q:<26}  →  {reply}")
print("=" * 58)
```

```
==========================================================
INPUT                        RESPONSE
==========================================================
  how are you                 →  i don't know .
  what is your name           →  <UNK> .
  where are you going         →  i'm going to get out .
  i love you                  →  you don't .
  what do you want            →  i .
  are you okay                →  i am .
  tell me a story             →  you don't have to .
  goodbye                     →  you are .
==========================================================
```

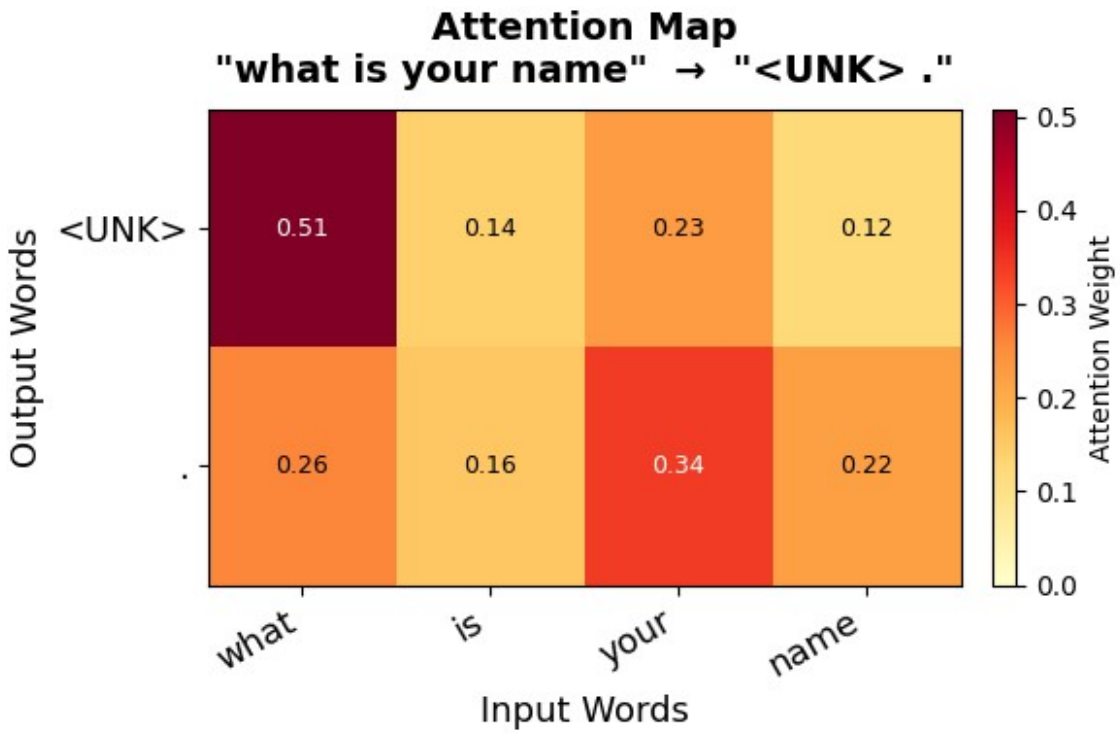
```
# ============================================================
# CELL 11: Attention heatmap — what the model focuses on
# ============================================================

def plot_attention(input_text, save=True):
    reply, attn_matrix, src_tokens = respond(input_text)
    tgt_tokens = reply.split()

    if not tgt_tokens or attn_matrix.shape[0] == 0:
        print("No output generated.")
        return

    # Trim attention to actual lengths
    attn = attn_matrix[:len(tgt_tokens), :len(src_tokens)]

    fig, ax = plt.subplots(figsize=(max(6, len(src_tokens)*1.2),
                                    max(4, len(tgt_tokens)*0.9)))
    im = ax.imshow(attn, cmap='YlOrRd', aspect='auto', vmin=0,
vmax=attn.max())

    ax.set_xticks(range(len(src_tokens)))
    ax.set_yticks(range(len(tgt_tokens)))
    ax.set_xticklabels(src_tokens, fontsize=13, rotation=30,
ha='right')
    ax.set_yticklabels(tgt_tokens, fontsize=13)
    ax.set_xlabel('Input Words',  fontsize=13)
    ax.set_ylabel('Output Words', fontsize=13)
```

```
    ax.set_title(f'Attention Map\n"{input_text}"  →  "{reply}"',
                 fontsize=14, fontweight='bold', pad=10)

    # Annotate cells
    for i in range(len(tgt_tokens)):
        for j in range(len(src_tokens)):
            ax.text(j, i, f"{attn[i,j]:.2f}", ha='center',
va='center',
                    fontsize=9, color='black' if attn[i,j] <
attn.max()*0.6 else 'white')

    plt.colorbar(im, ax=ax, fraction=0.03, pad=0.04, label='Attention
Weight')
    fig.tight_layout()

    if save:
        fname = f"{SAVE_DIR}/attn_{input_text.replace(' ','_')}.png"
        plt.savefig(fname, dpi=150)
        print(f"  □ Saved → {fname}")
    plt.show()
    return reply, attn

# Generate and save heatmaps for key examples
print("\n□ ATTENTION HEATMAPS\n")
for q in ["how are you", "what is your name", "i love you", "are you
okay"]:
    print(f"Input: '{q}'")
    result = plot_attention(q)
    if result:
        print(f"Response: '{result[0]}'\n")
```

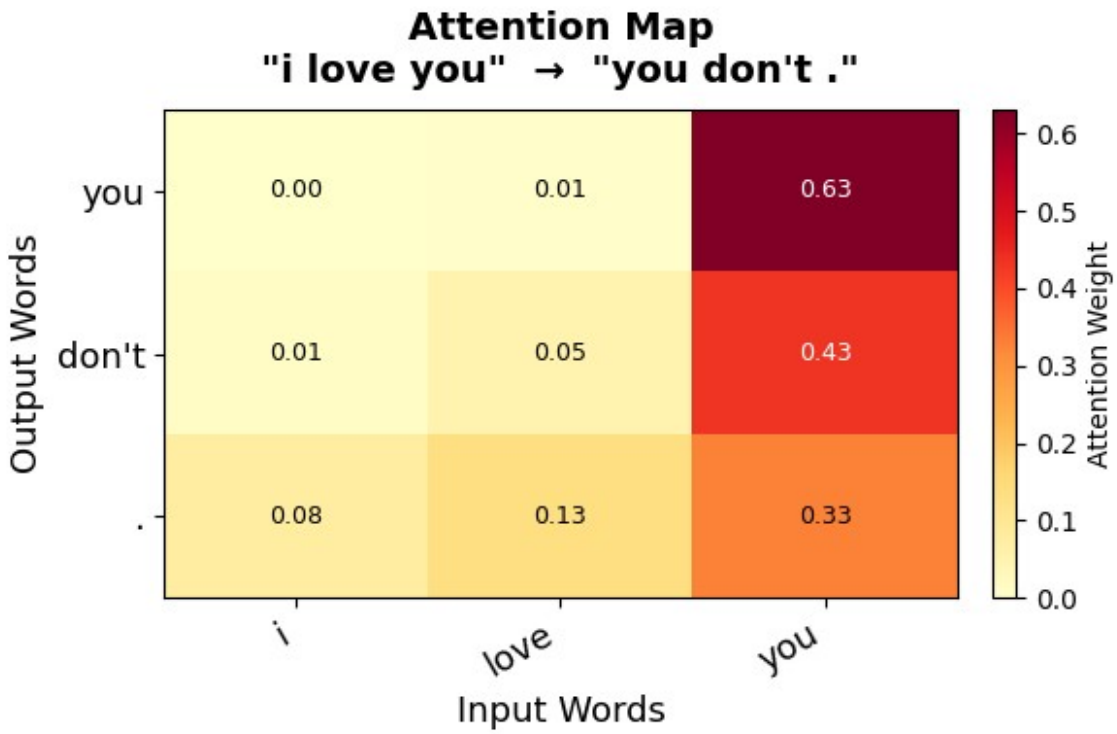
```
□ ATTENTION HEATMAPS

Input: 'how are you'
  □ Saved →
/content/drive/MyDrive/AttentionChatbot/attn_how_are_you.png
```



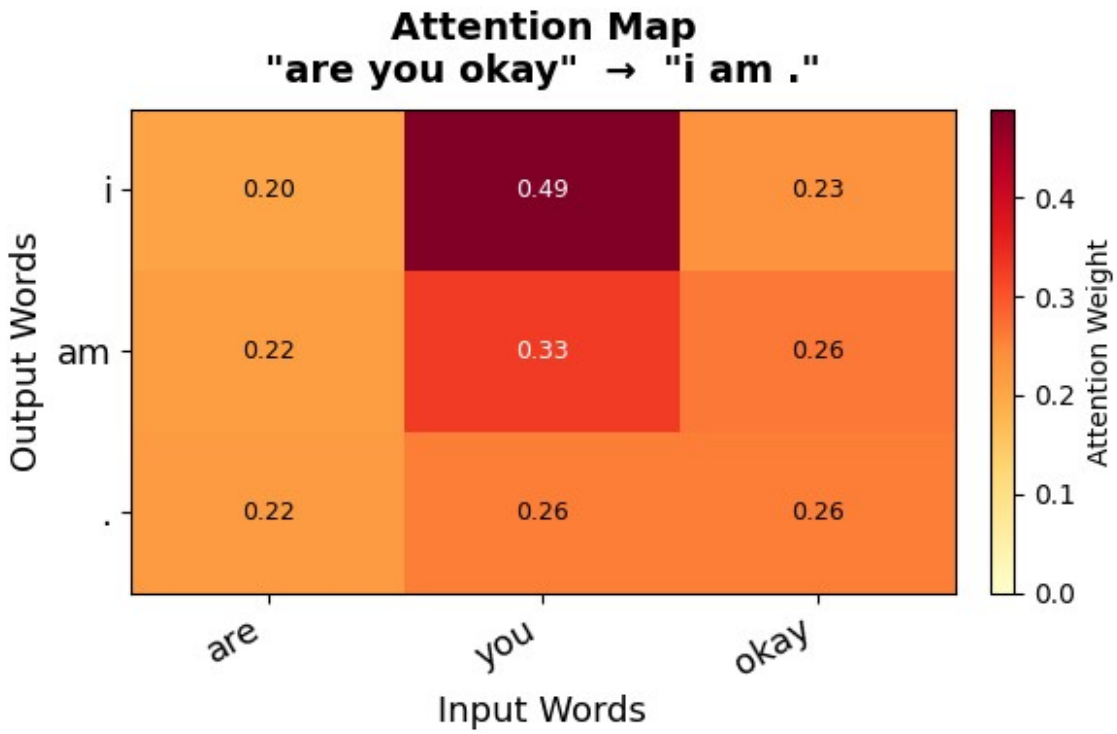

```
Response: 'i don't know .'

Input: 'what is your name'
  □ Saved →
/content/drive/MyDrive/AttentionChatbot/attn_what_is_your_name.png
```




```
Response: '<UNK> .'

Input: 'i love you'
  □ Saved →
/content/drive/MyDrive/AttentionChatbot/attn_i_love_you.png
```




```
Response: 'you don't .'

Input: 'are you okay'
  □ Saved →
/content/drive/MyDrive/AttentionChatbot/attn_are_you_okay.png
```




```
Response: 'i am .'


# ============================================================
# CELL 12: Save full results log to Drive
# ============================================================

results = []
for q in test_inputs:
    reply, attn_matrix, src_tokens = respond(q)
    # compute which input word got max attention (averaged across
output steps)
    if attn_matrix.shape[0] > 0 and len(src_tokens) > 0:
        avg_attn = attn_matrix.mean(axis=0)[:len(src_tokens)]
        focus_word = src_tokens[np.argmax(avg_attn)]
    else:
        focus_word = "N/A"
    results.append({
        "input": q,
        "response": reply,
        "most_attended_word": focus_word
    })

# Save as JSON
with open(f"{SAVE_DIR}/results.json", "w") as f:
    json.dump(results, f, indent=2)
```

```python
# Save as readable text report
with open(f"{SAVE_DIR}/results_report.txt", "w") as f:
    f.write("=" * 60 + "\n")
    f.write("ATTENTION CHATBOT — EVALUATION RESULTS\n")
    f.write("=" * 60 + "\n\n")
    f.write(f"Model: Seq2Seq + Bahdanau Attention\n")
    f.write(f"Vocab size: {VOCAB_SIZE:,}\n")
    f.write(f"Training pairs: {len(train_ds):,}\n")
    f.write(f"Best Val Loss: {best_val:.4f}\n\n")
    f.write("-" * 60 + "\n")
    for r in results:
        f.write(f"Input            : {r['input']}\n")
        f.write(f"Response         : {r['response']}\n")
        f.write(f"Top Attended Word: {r['most_attended_word']}\n\n")

print("□ All results saved to Drive!\n")
print("□ Files saved:")
for fname in os.listdir(SAVE_DIR):
    path = f"{SAVE_DIR}/{fname}"
    size = os.path.getsize(path) / 1024
    print(f"   {fname:40s}  {size:>8.1f} KB")
```

```
□ All results saved to Drive!

□ Files saved:
   vocab.json                                   136.4 KB
   best_model.pt                              87097.6 KB
   training_curves.png                           66.1 KB
   attn_how_are_you.png                          56.1 KB
   attn_what_is_your_name.png                    52.3 KB
   attn_i_love_you.png                           49.4 KB
   attn_are_you_okay.png                         47.1 KB
   results.json                                   0.8 KB
   results_report.txt                             1.0 KB
```

```python
# ============================================================
# CELL 13: Interactive chatbot (run in Colab)
# ============================================================

print("□ CHATBOT DEMO — type 'quit' to exit\n")
print("—" * 40)
while True:
    user_input = input("You: ").strip()
    if user_input.lower() in ('quit', 'exit', 'q'):
        print("Bot: Goodbye! □")
        break
    if not user_input:
        continue
    reply, attn, src_tokens = respond(user_input)
    print(f"Bot: {reply}")
```

```
    # Show top attended word
    if attn.shape[0] > 0 and len(src_tokens) > 0:
        avg_attn = attn.mean(axis=0)[:len(src_tokens)]
        focus_idx = np.argmax(avg_attn)
        print(f"     Model focused most on:
'{src_tokens[focus_idx]}' "
              f"(weight: {avg_attn[focus_idx]:.3f})")
    print()
```

```
 CHATBOT DEMO — type 'quit' to exit

────────────────────────────────────────
You: hi
Bot: hi .
     Model focused most on: 'hi' (weight: 0.100)

You: i know you
Bot: you don't know .
     Model focused most on: 'you' (weight: 0.490)

You: quit
Bot: Goodbye! 
```